Budgeted On-Call Queue for Cloud Microservice Operations

Aishah Mohamed Alyammahi
Student ID: 200010549
5 October 2026

1. Motivation

On-call engineers for a cloud platform receive more alerts each day than they can read. Standard practice ranks log messages by severity and removes exact duplicates, which misses faults that write no log line. On the public GAIA microservice dataset (10 services), 82% of injected faults never write a log line; they appear only as a jump in a memory metric. GAIA injects about 22 faults a day, so a budget of K = 20 items is already below the fault rate, and a fixed threshold either misses faults or admits more alerts than a person can review. This project asks: given a hard daily budget of K items, which items should the engineer see, using only information available at the moment of the decision?

2. Expected impact

The primary user is the on-call engineer, who receives one list of at most K items, each with a stated reason, instead of a stream of duplicate alerts. The platform team keeps control of paging: the service records what it would have paged and sends nothing until paging is explicitly enabled. Platform users benefit when a confirmed fault is shown after its second confirming observation. The queue ranks work for a person; it does not act on the fault. Replaying the rules on another team's metrics is optional, and only after the GAIA criteria in Section 4 are met.

3. Proposed approach

The Budgeted On-Call Queue will be a Python service: FastAPI exposes the queue and API, and SQLite stores alerts, jobs, and engineer marks. K is the maximum number of items the engineer is asked to read that day. Appendix A shows how the system moves from logs and memory to a list item or a page.

Detection combines three metric rules with a log classifier. Faults visible in a metric need plain rules, not a model: a memory jump is shown only after two confirming observations above the service's own recent median, so a single spike stays off the list; a slow-leak rule covers gradual climbs; a gap rule reports a series that stopped sending data. A model is used only where rules are weak, ranking log alerts, whose text, severity and timing interact: a cost-sensitive logistic regression in which a missed fault weighs about ten times one extra item. It is not used to find a fault that wrote no log. The queue is causal and first-come: an item may not use a later measurement, a jump's final peak, or the fault label.

Shadow mode is the default. Saved models are stored as a skops archive with a hash; a pickle file is refused. Optional Real or Noise marks may later retrain the log classifier. Evaluation uses the public GAIA dataset only; alerts from a real team would stay on that team's machine and be used only with its permission. Stack: Python, pandas, scikit-learn, FastAPI, SQLite, Docker, pytest. Training uses the first week only; later days are for scoring, and thresholds are fixed before that window. No paid service is required.

4. Deliverables and success criteria

The submission will include the web application and API, a command that regenerates the evaluation, automated tests, a Docker setup, a short design note, and a final report whose printed numbers match the result files. Success means all five checks below hold: a short list counts only if it stays inside K, uses no future information, catches more real faults than a rule a team already runs, and does not page anyone by accident.

Success criterion	How it will be judged
The daily budget holds	The queue never contains more than K items in one day. An automated test fails if it does.
Decisions use only past data	No item uses a later point, a jump's final peak, or the fault label. Scoring begins after the training week.
It beats a practical baseline	At K = 20, on the later days, the metric-jump detector must catch more faults than log severity and than a Prometheus-style rate rule (alert when memory rises by a fixed amount within minutes), with fewer false items. Severity alone is too weak because 82% of these faults write no log.
Paging stays off unless enabled	With shadow mode on, or without the password, the channel, and the word LIVE, no page is sent.
Another person can run it	The README installs the application with uv, and the test suite passes in continuous integration.

The core is the GAIA queue, the three metric rules, the log classifier, shadow mode, and these tests. Connectors for Slack or PagerDuty, a page for uploading another team's CSV, and a second public dataset come only after the criteria hold. Automatic repair of a fault and a hosted multi-company service are outside this project.

5. Timeline and feasibility

When	Milestone
Weeks 1–2	Load GAIA, align clocks to UTC, and score the severity baseline on the same time split.
Weeks 3–4	Implement the jump, leak, and gap rules, and train the log classifier on the first week only.
Weeks 5–6	Enforce the daily budget and add tests that the queue never looks ahead.
Weeks 7–8	Complete the web application, shadow mode, Docker, the report, and a short demonstration.

The dataset is public and the tools are standard. Material risks: every GAIA fault is the same class of memory injection, so a strong score may not transfer (the report will state this); a new team has no labels on day one, so the metric rules must work before any classifier is trained; and logs and metrics use different clocks, so evaluation will use one shared UTC clock.

Appendix A. How the Budgeted On-Call Queue decides what to show

Read the figure from top to bottom. Steps 4a to 4d can all find a problem. Step 5 keeps only the earliest items, and never more than K. A page is sent only at step 7.

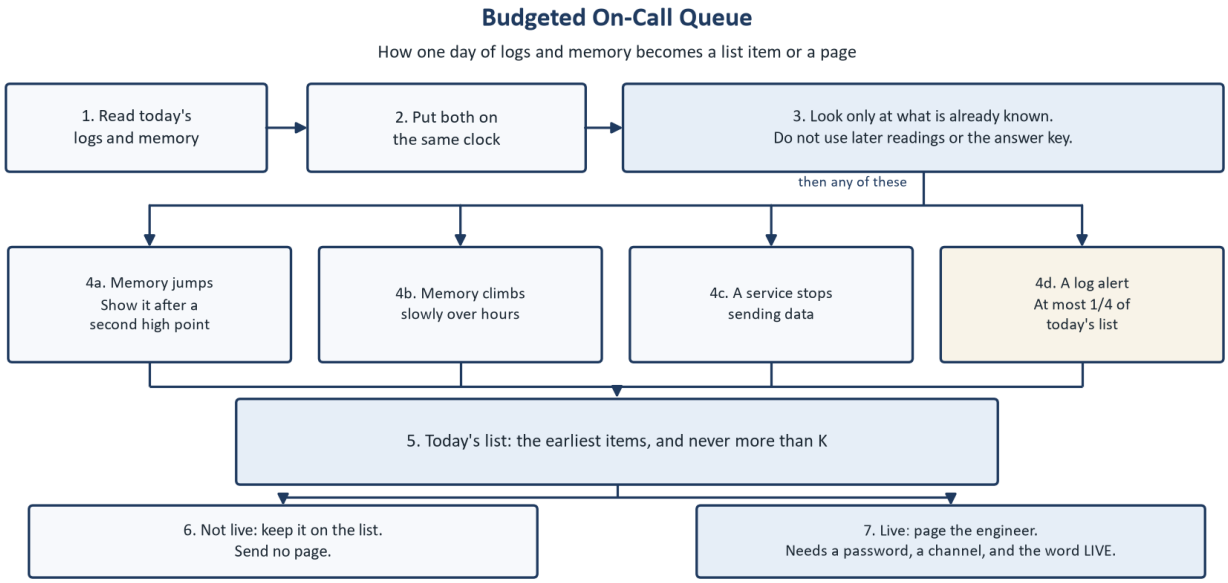


Figure A1. The Budgeted On-Call Queue. The shaded step is the rule that blocks future information. The cream box is the only step that reads logs, and it may fill at most one quarter of the list.